KYAMBOGO UNIVERSITY

FACULTY OF ENGINEERING

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

TETE 3102: MICROCONTROLLERS AND APPLICATIONS

COURSEWORK / MINI-PROJECT REPORT

DESIGN AND IMPLEMENTATION OF A MULTI-TASK REAL-TIME APPLICATION ON STM32 USING FreeRTOS

A Temperature Monitoring and Control Case Study

SUBMITTED BY (GROUP MEMBERS):

Student Name	Registration Number
Mukama Robert	26/U/ETE/21285/PE
Nabudere Zerubabel	25/U/ETE/05465/PE
Nanfuka Shakirah	25/U/ETE/05468/PE
Kidega Wilkie Cephas	25/U/ETE/05460/PE

SUPERVISED BY:

Dr Mugerwa Dickson PhD

Department of Electrical and Electronics Engineering

A report submitted to the Department of Electrical and Electronics Engineering, Faculty of Engineering, Kyambogo University, in partial fulfilment of the requirements for the course **TETE 3102: Microcontrollers and Applications** for the award of the degree of

Bachelor of Engineering in Electrical and Electronics Engineering

Date of Submission: October 2026

Declaration

We, the undersigned group members, declare that this coursework report is our own original work and has not been submitted for any academic award in this or any other institution. All sources of information and literature references have been duly acknowledged.

Student Name	**Registration Number**	**Signature & Date**
Mukama Robert	26/U/ETE/21285/PE	.....
Nabudere Zerubabel	25/U/ETE/05465/PE	.....
Nanfuka Shakirah	25/U/ETE/05468/PE	.....
Kidega Wilkie Cephas	25/U/ETE/05460/PE	.....

This report has been submitted with my approval as the course lecturer and supervisor.

....

Dr Mugerwa Dickson PhD **Date**

(Course Lecturer / Supervisor)

Abstract

A Real-Time Operating System (RTOS) guarantees that critical tasks complete within deterministic, bounded time constraints—a property that a bare-metal super-loop cannot provide once an application grows beyond a handful of concurrent activities. This report presents the theory, configuration and practical design of a multi-task real-time application implemented on an STM32 Cortex-M microcontroller using FreeRTOS. The core RTOS concepts of tasks, preemptive priority scheduling, task states, context switching, inter-task communication (queues, semaphores, mutexes, event flags), synchronisation and memory management are reviewed and then applied. A temperature monitoring and control system comprising four periodic tasks—button/UI (priority 4), sensor acquisition (priority 3), control actuation (priority 2) and UART logging (priority 1)—was designed, with communication handled through a queue, a mutex, a binary semaphore and an event group. Rate-monotonic analysis yields a processor utilisation of $U = 0.126$, well below the $n = 4$ schedulability bound of 0.757, proving the task set meets all deadlines. The report concludes that FreeRTOS on STM32 delivers deterministic, modular and scalable concurrency, and that its principal challenges—priority inversion, stack overflow and race conditions—are manageable through disciplined use of mutexes, static allocation and runtime stack monitoring.

Keywords: RTOS, FreeRTOS, STM32, Cortex-M, STM32CubeMX, real-time scheduling, task prioritisation, semaphore, mutex, priority inversion.

Contents

Declaration i

Abstract ii

List of Tables v

List of Figures vi

List of Code Listings vii

List of Abbreviations viii

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 General-Purpose OS vs Bare-Metal vs RTOS . . . 1
  - 1.2.1 Bare-Metal Systems (Super-Loop and Foreground-Background) . . . 1
  - 1.2.2 General-Purpose Operating Systems (GPOS) . . . 4
  - 1.2.3 Real-Time Operating Systems (RTOS) . . . 5
  - 1.2.4 Architectural Synthesis and Paradigm Selection Framework . . . 7
- 1.3 Importance of RTOS in Real-Time Embedded Systems . . . 7

2 Objectives 9

3 Theory 10
- 3.1 Tasks and Scheduling . . . 10
- 3.2 Task States and Context Switching . . . 10
- 3.3 Inter-Task Communication and Synchronisation . . . 10
- 3.4 Memory Management . . . 11
- 3.5 Schedulability Analysis . . . 11
- 3.6 FreeRTOS on STM32 . . . 11

4 Materials and Methods 12
- 4.1 Equipment and Software . . . 12

4.2 Integration of FreeRTOS with STM32CubeMX and STM32CubeIDE . . . 12
4.3 Design Methodology for the Multi-Task Application 13

5 Results **14**
5.1 System Overview 14
5.2 Tasks, Priorities and Timing 14
5.3 Inter-Task Communication and Synchronisation 14
5.4 Schedulability Analysis 15
5.5 Comparison with a Bare-Metal Implementation 15

6 Discussion **16**
6.1 Interpretation of Results 16
6.2 Advantages of Using an RTOS on STM32 16
6.3 Challenges and Pitfalls 16
6.4 When Not to Use an RTOS 17
6.5 Sources of Error and Limitations 17

7 Conclusion and Recommendations **18**
7.1 Conclusion 18
7.2 Recommendations 18

References **19**

A Sample Rate-Monotonic Calculation **20**

B FreeRTOS Task Source Code **21**

C Task Creation Snippet **24**

D Datasheets and Additional Resources **25**

List of Tables

1.1 Comprehensive comparison of Bare-Metal, General-Purpose OS (GPOS), and Real-Time OS (RTOS). 2

4.1 Hardware and software used in the experiment. 12

5.1 Task set: functions, priorities, periods, worst-case execution times and stack sizes. 14

5.2 Comparison of the RTOS design with an equivalent bare-metal super-loop implementation. 15

List of Figures

List of Code Listings

B.1 Temperature sensor task (T2) — producer. 21
B.2 Control task (T3) — consumer and actuator driver. 21
B.3 Logging task (T4) — lowest priority, protected by a mutex. 22
B.4 ADC interrupt callback — deferred processing via a binary semaphore. . . 23
C.1 Task and primitive creation inside `MX_FREERTOS_Init()`. 24

List of Abbreviations

Abbreviation	Meaning
ADC	Analogue-to-Digital Converter
API	Application Programming Interface
CMSIS	Cortex Microcontroller Software Interface Standard
DWT	Data Watchpoint and Trace
GPIO	General-Purpose Input/Output
GPOS	General-Purpose Operating System
HAL	Hardware Abstraction Layer
IDE	Integrated Development Environment
ISR	Interrupt Service Routine
ITC	Inter-Task Communication
MCU	Microcontroller Unit
MISRA	Motor Industry Software Reliability Association
PLC	Programmable Logic Controller
RM	Rate Monotonic
RTOS	Real-Time Operating System
SWD	Serial Wire Debug
UART	Universal Asynchronous Receiver/Transmitter
WCET	Worst-Case Execution Time

Chapter 1

Introduction

1.1 Background

A **Real-Time Operating System (RTOS)** is a specialised operating system kernel designed to manage hardware resources and execute application tasks such that critical operations complete within *deterministic, bounded* time constraints. Unlike a general-purpose operating system (GPOS) whose primary goal is average throughput and fairness, an RTOS guarantees that a high-priority task will preempt lower-priority work and respond within a known worst-case latency.

The purpose of an RTOS is therefore threefold: (i) to provide **deterministic scheduling** of multiple concurrent activities, (ii) to abstract hardware through **task, queue, semaphore and mutex primitives**, and (iii) to enable **modular, maintainable firmware** whose timing behaviour can be analysed and proven before deployment.

1.2 General-Purpose OS vs Bare-Metal vs RTOS

In embedded and computer systems engineering, selecting the appropriate software execution paradigm is a foundational design decision that directly dictates system determinism, development complexity, resource footprint, and runtime reliability. Systems are broadly classified into three architectural paradigms: **Bare-Metal** (Super-loop), **General-Purpose Operating Systems (GPOS)**, and **Real-Time Operating Systems (RTOS)**. Table 1.1 provides a comprehensive comparative analysis across key structural and operational dimensions.

1.2.1 Bare-Metal Systems (Super-Loop and Foreground-Background)

The bare-metal model represents the traditional foundation of microcontroller firmware. In this paradigm, the CPU executes firmware instructions directly on the physical hardware

Table 1.1: Comprehensive comparison of Bare-Metal, General-Purpose OS (GPOS), and Real-Time OS (RTOS).

Criterion	Bare-Metal (Super-Loop)	General-Purpose OS	Real-Time OS (RTOS)
Execution Architecture	Sequential `while(1)` polling loop interleaved with hardware ISRs	Multi-tier kernel with user/kernel space separation and virtual memory	Compact microkernel with independent thread/task stacks
Scheduling Policy	Cyclic sequential execution; manual state-machine sequencing	Fair-share, time-sliced, throughput-optimised (e.g. Linux CFS)	Deterministic, priority-based preemptive scheduling (Rate-Monotonic)
Temporal Determinism	Low; execution latency fluctuates with loop workload	Non-deterministic; high jitter due to paging, daemons, and caching	Strictly bounded and analysable; guaranteed worst-case execution latency
Context Switch Overhead	None (ISR context saving only, sub-microsecond)	Significant (tens of microseconds to milliseconds; MMU/TLB flush)	Minimal and bounded (1–3 μs on Cortex-M via PendSV)
Interrupt Latency	Minimal hardware vector latency ($\sim$ 12 clock cycles)	Unpredictable; delayed by kernel critical sections and paging	Minimal and bounded; direct hardware interrupt preemption
Memory Footprint	Extremely low ($<$ 1 KB RAM, few KB Flash)	Substantial (hundreds of megabytes to gigabytes of RAM/storage)	Highly compact (4–10 KB Flash, 2–5 KB RAM kernel overhead)
Hardware Requirements	Basic 8/16/32-bit MCU; no MMU or cache required	High-performance MPU with Memory Management Unit (MMU)	16/32-bit MCU (e.g. Cortex-M); MMU optional, optional MPU
Representative Software	Raw CMSIS C, STM32 HAL/LL, Arduino Core, Microchip Harmony	Linux (Ubuntu, Debian, Yocto), Android, Windows 10/11, macOS	FreeRTOS, Zephyr RTOS, VxWorks, QNX Neutrino, RTEMS, μC/OS-III
Real-World Applications	Microwave oven, digital thermometer, TV remote, mouse	Smartphone, ATM, industrial HMI, car infotainment, cloud server	Automotive ABS/ECU, flight avionics, pacemaker, robotics, PLC

without any mediating operating system abstraction layer. The architecture typically consists of two distinct components:

1. **Background (Super-Loop):** An infinite polling loop (`while(1)`) inside the `main()` routine that sequentially scans flags, executes application subroutines, and updates peripherals.
2. **Foreground (Interrupt Service Routines):** Asynchronous hardware interrupts triggered by external timers, pin transitions, or communication peripherals that urgently service hardware events and set volatile flags for the background loop.

Representative Software and Frameworks: Bare-metal firmware is typically developed using low-level Hardware Abstraction Layers or direct register manipulation:

- **CMSIS Core:** Direct register manipulation in C/C++ on ARM Cortex-M processors.
- **Vendor HAL/LL Libraries:** STMicroelectronics STM32 HAL/LL in standalone non-RTOS mode, Microchip MPLAB XC8/XC32, and Texas Instruments DriverLib.
- **Arduino Core:** The standard `setup()` and `loop()` execution model executed across 8-bit AVR (e.g. ATmega328P) and SAMD architectures.

Concrete Real-Life Examples: Bare-metal architecture is optimal for dedicated, cost-constrained products with singular, straightforward duties:

- **Digital Clinical Thermometer:** Reads an analogue thermistor upon a button press, performs ADC conversion, displays the temperature on a 7-segment LCD, and enters low-power sleep.
- **Microwave Oven Keypad Controller:** Continually scans a 4×4 matrix keypad, counts down a hardware timer, and drives a buzzer and magnetron relay.
- **Infrared Television Remote Control:** Rests in deep sleep until a button matrix generates an interrupt, transmits a pulse-width modulated 38 kHz IR burst, and immediately returns to sleep.
- **Optical Computer Mouse:** Reads an optical sensor via SPI, debounces left/right click microswitches, and transmits USB HID packets.

Engineering Limitations: While bare-metal design provides zero OS footprint and simple debugging, it breaks down rapidly as application complexity scales. Because all functions share one thread of execution, introducing a blocking call (such as waiting 20 ms for an I2C sensor or formatting a serial string) directly delays every other function in the loop.

The timing jitter of any individual task becomes coupled to the cumulative execution time of all other tasks, making deterministic guarantees impossible without fragile, nested manual state machines.

1.2.2 General-Purpose Operating Systems (GPOS)

A General-Purpose Operating System is an expansive, multi-user, multi-process software platform designed to manage complex hardware subsystems, maximize aggregate computational throughput, and ensure fair resource allocation across diverse concurrent applications. GPOS platforms rely heavily on a hardware Memory Management Unit (MMU) to provide virtual memory address spaces, ensuring process isolation and preventing runaway user programs from crashing the kernel.

Representative Software Stacks:

- **Linux Distributions:** Canonical Ubuntu, Debian GNU/Linux, Red Hat Enterprise Linux, and customized embedded Linux built via the Yocto Project or Buildroot.
- **Mobile and Consumer OS:** Google Android (AOSP), Apple iOS and macOS.
- **Desktop and Enterprise OS:** Microsoft Windows 10/11 and Windows 10 IoT Enterprise.

Concrete Real-Life Examples: GPOS solutions excel in high-level computing environments characterized by rich graphical user interfaces, extensive networking stacks, file storage, and non-time-critical human interaction:

- **Automated Teller Machines (ATMs) and POS Terminals:** Typically powered by Windows 10 IoT or Linux, managing bank network communication, graphical customer prompts, touchscreens, and encrypted transaction logging.
- **Smartphones and Tablets:** Running Android or iOS, concurrently orchestrating cellular modems, Wi-Fi/Bluetooth stacks, camera image pipelines, and third-party applications.
- **Automotive In-Vehicle Infotainment (IVI):** Android Automotive or Linux driving multi-screen center consoles, GPS satellite navigation, and multimedia streaming.
- **Industrial Human-Machine Interfaces (HMIs):** High-resolution touch panels running Linux/Qt for process supervision, plant floor SCADA visualization, and database historian logging.

Incompatibility with Hard Real-Time Deadlines: Despite their immense computing power, GPOS kernels are inherently unsuitable for hard real-time physical control. Schedulers such as the Linux Completely Fair Scheduler (CFS) allocate time slices based on dynamic fairness heuristics rather than strict event criticality. Dynamic memory paging, translation lookaside buffer (TLB) misses, background maintenance daemons, disk cache flushes, and kernel preemption latencies introduce timing jitter in the tens of milliseconds. In a motor control loop or airbag deployment circuit, an unpredictable 10 ms stall while the kernel services virtual memory causes catastrophic system failure.

1.2.3 Real-Time Operating Systems (RTOS)

A Real-Time Operating System is a lean, deterministic software kernel engineered specifically to guarantee that high-priority computing tasks complete within rigorous, mathematically bounded deadlines. The defining metric of an RTOS is not computational speed or average throughput, but **predictability and temporal determinism**.

An RTOS achieves determinism through:

- **Preemptive Priority Scheduling:** The scheduler continuously ensures that the highest-priority task currently in the *Ready* state is granted the CPU immediately, preempting lower-priority execution within microseconds.
- **Deterministic Context Switching:** Register saving and restoring is bounded and optimized (typically sub-3 μs on Cortex-M microcontrollers via the hardware PendSV exception).
- **Disciplined Inter-Task Communication (ITC):** Queues, semaphores, event groups, and priority-inheritance mutexes prevent data corruption and eliminate unbounded priority inversion.

Real-time systems are universally classified into two functional domains:

1. **Hard Real-Time Systems:** Systems where missing a single timing deadline constitutes a catastrophic failure, resulting in physical destruction, environmental damage, or loss of human life.
2. **Soft Real-Time Systems:** Systems where missing a deadline degrades the quality of service (QoS) or user experience but does not cause physical harm or total system breakdown.

Representative Software Stacks:

- **Open-Source and Permissive Kernels:**

  - **FreeRTOS:** The de-facto industry standard for microcontrollers, maintained by Amazon Web Services and natively supported by STMicroelectronics, NXP, and Espressif. Features a kernel footprint under 10 KB.
  - **Zephyr RTOS:** A modern, scalable open-source RTOS hosted by the Linux Foundation, engineered for connected IoT devices with built-in networking and modular device drivers.
  - **Apache NuttX and RT-Thread:** POSIX-compliant open-source RTOS kernels for resource-constrained microcontrollers.

- **Safety-Certified Commercial Kernels:**
  - **Wind River VxWorks:** A mission-critical RTOS deployed in aerospace, defense, and space exploration (e.g. the NASA Mars Curiosity and Perseverance rovers, Boeing 787 avionics).
  - **BlackBerry QNX Neutrino:** A POSIX microkernel certified to ISO 26262 ASIL-D for automotive safety and IEC 61508 SIL-3 for industrial systems, widely used in advanced driver-assistance systems (ADAS) and digital instrument clusters.
  - **Green Hills INTEGRITY:** Certified to FAA DO-178C Level A for flight-critical commercial avionics.
  - **Micrium μC/OS-III:** A deterministic, safety-certified commercial kernel popular in industrial medical devices.

Concrete Real-Life Examples:

- **Automotive Anti-lock Braking Systems (ABS) and Airbag ECUs (Hard Real-Time):** Wheel speed sensors are sampled at kilohertz rates. If wheel lockup is detected, hydraulic solenoid valves must modulate brake pressure within microseconds. Airbag deployment algorithms must execute and trigger pyrotechnic actuators within 15–30 ms of collision detection; a late response is fatal.
- **Implantable Cardiac Pacemakers and Infusion Pumps (Hard Real-Time):** Cardiac electrical signals are sensed continuously; pacing electrical impulses must be delivered with zero microsecond-level timing jitter. Intravenous drug infusion pumps must accurately meter dosage rates without software lockups.
- **Aerospace Flight Control and Satellite Attitude Control (Hard Real-Time):** Quadrotor drone flight controllers and commercial fly-by-wire flight control computers run high-rate inertial measurement unit (IMU) filtering and PID stabilization loops every 1–5 ms. Any timing stall results in immediate aerodynamic instability.

- **Digital Audio Streamers and Voice-over-IP (Soft Real-Time):** Audio packets must be processed and passed to an I2S DAC buffer at steady sampling rates (e.g. 48 kHz). If a deadline is missed due to high network traffic, an audible click or drop-out occurs, degrading acoustic quality without causing device destruction.

1.2.4 Architectural Synthesis and Paradigm Selection Framework

An embedded systems engineer must select between Bare-Metal, RTOS, and GPOS by evaluating functional complexity, timing criticality, and hardware constraints:

- **Select Bare-Metal when:** The application is dedicated to a single repetitive duty, has no hard concurrent timing deadlines, operates on an ultra-low-cost MCU (< 4 KB RAM), and development time is minimal.
- **Select an RTOS (e.g. FreeRTOS on STM32) when:** The system must perform multiple asynchronous or periodic activities with hard timing guarantees (e.g. sensing, control loop, user input, and communication logging), requires modular maintainability, and operates on an ARM Cortex-M microcontroller with 16–512 KB of RAM.
- **Select a GPOS when:** The product requires rich graphical user interfaces, multi-gigabyte databases, high-speed multi-protocol networking, and third-party application hosting on high-performance Cortex-A or x86 processors without hard microsecond deadline constraints.
- **Modern Hybrid Paradigm (AMP Architecture):** In complex modern systems, engineers frequently combine GPOS and RTOS using Asymmetric Multiprocessing (AMP) SoCs (e.g. STM32MP1 or NXP i.MX8). On such platforms, an ARM Cortex-A core executes Linux for the touchscreen GUI and cloud connectivity, while a companion ARM Cortex-M core concurrently runs FreeRTOS to manage real-time motor actuation and safety monitoring with guaranteed determinism.

1.3 Importance of RTOS in Real-Time Embedded Systems

Real-time systems are classified as **hard** (missing a deadline is a system failure) or **soft** (missing a deadline degrades quality of service). Examples where an RTOS is essential include:

- **Industrial control:** PLCs and motor drives must actuate within microseconds to prevent mechanical damage.
- **Automotive:** ABS, airbag deployment and engine management are hard real-time; a late airbag trigger is fatal.

- **IoT:** sensor nodes must sample, timestamp and transmit data with bounded latency to preserve data integrity.
- **Medical devices:** pacemakers and infusion pumps must deliver therapy at exact intervals; jitter is clinically unacceptable.

Chapter 2

Objectives

The objectives of this assignment are:

1. To define an RTOS and distinguish it from bare-metal and general-purpose operating systems.
2. To examine the core RTOS concepts: tasks, scheduling, task states, context switching, inter-task communication, synchronisation and memory management.
3. To investigate how FreeRTOS is integrated with STM32 microcontrollers using STM32-CubeMX and STM32CubeIDE.
4. To identify the advantages and challenges of migrating from bare-metal firmware to an RTOS, and to state when an RTOS should *not* be used.
5. To **design a multi-task STM32 application** (a temperature monitoring and control system), specifying the number of tasks, their priorities, their communication and synchronisation mechanisms, and justifying the RTOS approach over bare-metal.

Chapter 3

Theory

3.1 Tasks and Scheduling

An RTOS is built around the concept of a **task**: an independent thread of execution with its own stack and program counter. Because only one task can run on a single-core Cortex-M at any instant, the kernel uses a **scheduler** to decide which task executes. The dominant policies are **preemptive priority-based scheduling** (the highest-priority ready task always runs), **cooperative scheduling** (a task yields voluntarily) and **round-robin** (equal-priority tasks share time slices). FreeRTOS on STM32 uses preemptive priority scheduling with optional time-slicing.

3.2 Task States and Context Switching

Each task exists in one of four **states**: *Running*, *Ready*, *Blocked* (waiting for a delay, queue or semaphore) or *Suspended*. Transitions between these states are driven by the scheduler and by kernel API calls. When a higher-priority task becomes ready, the kernel performs a **context switch**: it saves the current task's CPU registers onto its stack, restores the registers of the next task and resumes execution. On ARM Cortex-M this is accelerated by the hardware **PendSV** exception and the SysTick timer.

3.3 Inter-Task Communication and Synchronisation

Tasks must exchange data without corrupting shared memory. An RTOS provides **inter-task communication (ITC)** primitives:

- **Queues** — FIFO data transfer between tasks and from ISRs.
- **Semaphores** — binary and counting signalling.
- **Mutexes** — mutual exclusion with priority inheritance.

- **Event flags / event groups** — bitwise synchronisation of multiple conditions.

For shared resources, a **mutex** prevents simultaneous access and mitigates **priority inversion** through inheritance, bounding the classic three-task inversion scenario.

3.4 Memory Management

Memory management in an RTOS concerns the per-task **stack** (statically sized at creation; overflow is the most common failure mode) and the **heap** (managed by `heap_1` to `heap_5` schemes in FreeRTOS). Static allocation is preferred in safety-critical designs because it removes fragmentation and runtime allocation failure.

3.5 Schedulability Analysis

The schedulability of a periodic task set under Rate Monotonic (RM) scheduling is checked with the utilisation bound:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq n\left(2^{1/n} - 1\right) \tag{3.1}$$

where C_i is the worst-case execution time and T_i the period of task i. If U satisfies the bound in Equation (3.1), all deadlines are guaranteed.

3.6 FreeRTOS on STM32

FreeRTOS is the most widely used RTOS on STM32 because it is open-source, MISRA-friendly and natively integrated into ST's toolchain. The kernel footprint can be under 6 KB, making it suitable even for low-end Cortex-M0 devices. Key configuration parameters are exposed in `FreeRTOSConfig.h`: `configTICK_RATE_HZ` (typically 1000 Hz, i.e. a 1 ms tick), `configMAX_PRIORITIES`, `configUSE_PREEMPTION`, `configTOTAL_HEAP_SIZE` and `configUSE_MUTEXES`.

Chapter 4

Materials and Methods

4.1 Equipment and Software

Table 4.1: Hardware and software used in the experiment.

Item	Description
Microcontroller board	STM32 Nucleo / Discovery (Cortex-M4, e.g. STM32F411RE)
Sensor	LM35 analogue temperature sensor (or DHT22)
Actuators	Status LED; DC fan or heater driven through a relay/MOSFET
Input device	Push button for user interaction
Interface	UART–USB bridge for serial logging
IDE	STM32CubeIDE v1.15 or later
Configurator	STM32CubeMX (integrated into CubeIDE)
RTOS	FreeRTOS via the CMSIS-RTOS v2 wrapper (bundled)
Debugger	ST-LINK/V2 with SWD
Instrumentation	Oscilloscope / logic analyser; DWT cycle counter

4.2 Integration of FreeRTOS with STM32CubeMX and STM32CubeIDE

The configuration workflow followed in this work is:

1. **Create a project** in STM32CubeIDE and select the target MCU.
2. **Configure peripherals** (ADC for the sensor, GPIO for the LED and button, USART2 for logging) in the graphical pinout.
3. **Enable FreeRTOS** under *Middleware* → *FREERTOS*, selecting the CMSIS-RTOS v2 API.
4. **Add tasks** in the *Tasks and Queues* tab (name, priority, stack size, entry function).

5. **Add queues, mutexes, semaphores and event groups** in the same tab.
6. **Generate code:** CubeMX automatically generates `MX_FREERTOS_Init()`, the task stubs and `freertos.c`.
7. **Insert application logic** inside the generated task functions and rebuild.

This removes the need to manually port `port.c`, `FreeRTOSConfig.h` and the SysTick-/PendSV handler mapping.

4.3 Design Methodology for the Multi-Task Application

A **temperature monitoring and control system** was designed. The system reads temperature periodically, drives a fan or heater according to thresholds, updates an LED indicator, logs over UART and responds to a user button. The design followed five steps: (i) identify concurrent activities, (ii) assign priorities by criticality and rate, (iii) select communication primitives, (iv) perform rate-monotonic schedulability analysis, and (v) compare the result against a bare-metal equivalent.

Chapter 5

Results

5.1 System Overview

The application comprises **four periodic tasks** coordinated by a queue, a mutex, a binary semaphore and an event group, all running under FreeRTOS on the STM32.

5.2 Tasks, Priorities and Timing

Table 5.1: Task set: functions, priorities, periods, worst-case execution times and stack sizes.

Task	Function	Priority	T_i (ms)	C_i (ms)	Stack (words)
T1 Button/UI	Debounce button; set event flags	4 (highest)	50	2	128
T2 Sensor	Read ADC/LM35; push to queue	3	100	5	256
T3 Control	Compare threshold; drive fan + LED	2	200	4	256
T4 Logger	Format and transmit UART string	1 (lowest)	500	8	512

5.3 Inter-Task Communication and Synchronisation

- **Queue (`xQueueTemp`):** T2 sends the sampled temperature to T3. This decouples the producer from the consumer and enforces data consistency.
- **Mutex (`xUartMutex`):** guards the shared USART peripheral so that T1 (menu echo) and T4 (logging) never interleave their output.
- **Event group (`xEventGroup`):** T1 sets `EVT_BUTTON_PRESS`; T3 waits on it to toggle

between manual and automatic mode. This is a lightweight, non-blocking synchronisation mechanism.

- **Binary semaphore (`xAdcDone`):** signalled by the ADC interrupt service routine and taken by T2, implementing deferred interrupt processing via `xSemaphoreGiveFromISR()`.

5.4 Schedulability Analysis

Using the periods and execution times of Table 5.1:

$$U = \frac{2}{50} + \frac{5}{100} + \frac{4}{200} + \frac{8}{500} = 0.04 + 0.05 + 0.02 + 0.016 = 0.126 \tag{5.1}$$

The RM bound for $n = 4$ is $4\left(2^{1/4} - 1\right) = 0.757$. Since $U = 0.126 < 0.757$, the task set is **guaranteed schedulable** under rate-monotonic priority assignment. This confirms that the priority ordering $4 \rightarrow 1$ (fastest and most critical task highest) is correct.

5.5 Comparison with a Bare-Metal Implementation

Table 5.2: Comparison of the RTOS design with an equivalent bare-metal super-loop implementation.

Aspect	Bare-Metal Equivalent	RTOS Design
Temperature sampling	Delayed by UART string formatting (≈8 ms)	Always meets the 100 ms deadline
Button response	Polled once per loop iteration	Preempts within approximately one tick (1 ms)
Adding a new task	Requires restructuring the loop	Add one task; no other code changes
Timing verification	Empirical only	Analytically provable (Equation (3.1))
CPU idle time	Busy-wait	Idle task enters low-power mode

Chapter 6

Discussion

6.1 Interpretation of Results

The utilisation of 12.6 % obtained in Equation (5.1) is far below the 75.7 % RM bound, meaning the CPU has substantial headroom and could absorb additional tasks (for example wireless transmission) without violating deadlines. The priority ordering (button $>$ sensor $>$ control $>$ logger) reflects the *criticality-and-rate* rule: the button demands the fastest human-perceptible response, while logging tolerates the greatest latency and is therefore relegated to the lowest priority, where it cannot disturb the control loop.

6.2 Advantages of Using an RTOS on STM32

- **Determinism:** every task has a bounded response time, verified analytically.
- **Modularity:** each activity is an isolated function with its own stack, so teams can develop in parallel.
- **Scalability:** new features are added as tasks without destabilising existing timing.
- **Reusability:** queues, mutexes and semaphores are standard, portable primitives.
- **Power efficiency:** when all tasks are blocked, the idle task can enter a low-power sleep state.

6.3 Challenges and Pitfalls

- **Priority inversion:** mitigated with mutex priority inheritance, but poorly designed code can still deadlock.
- **Stack overflow:** the most common RTOS crash; requires `configCHECK_FOR_STACK_OVERFLOW` and conservative sizing.

- **Race conditions:** unguarded shared data corrupts silently; all shared resources must be protected.
- **Increased complexity:** more code paths and greater debugging difficulty than a simple super-loop.
- **ISR discipline:** only `...FromISR()` APIs may be called from interrupts, and they must be short.
- **Tick overhead:** a 1 kHz tick consumes CPU cycles; tickless idle is required for battery-powered devices.

6.4 When Not to Use an RTOS

An RTOS is unnecessary when the application is (i) a single, simple function, (ii) extremely RAM- or Flash-constrained (for example an 8-bit MCU with 2 KB of RAM), (iii) hard real-time at the hardware level where a dedicated peripheral or interrupt suffices, or (iv) so trivial that a super-loop meets all deadlines with ample margin. In these cases the RTOS adds overhead without benefit.

6.5 Sources of Error and Limitations

The WCET figures in Table 5.1 are estimates rather than measured worst cases; actual values depend on compiler optimisation, flash wait states and interrupt load. The analysis also assumes independent periodic tasks, whereas real ISRs and shared buses introduce jitter. Future work should measure execution times with the DWT cycle counter and verify stack high-water marks at runtime.

Chapter 7

Conclusion and Recommendations

7.1 Conclusion

This report defined the RTOS and distinguished it from bare-metal and general-purpose systems; examined core concepts including scheduling, task states, context switching, inter-task communication, synchronisation and memory management; and described FreeRTOS integration with STM32CubeMX/CubeIDE. A four-task temperature monitoring and control application was designed, with priorities assigned by rate monotonic policy and a computed utilisation of 12.6 % confirming schedulability. The objectives were fully achieved: the design demonstrates that an RTOS provides deterministic, modular and scalable concurrency that a bare-metal super-loop cannot guarantee, while its challenges are manageable through disciplined use of mutexes, queues and stack monitoring.

7.2 Recommendations

1. Measure rather than estimate WCET values using the DWT cycle counter on the STM32.
2. Enable `configCHECK_FOR_STACK_OVERFLOW` and `configUSE_TRACE_FACILITY` during development.
3. Prefer static allocation (`xTaskCreateStatic`) in safety-critical designs.
4. Use mutexes with priority inheritance for all shared peripherals.
5. Adopt tickless idle mode for battery-operated IoT nodes.

Bibliography

[1] R. Barry, *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd., 2016.

[2] P. A. Laplante, *Real-Time Systems Design and Analysis*, 4th ed. Hoboken, NJ: Wiley-IEEE Press, 2011.

[3] J. W. S. Liu, *Real-Time Systems*. Upper Saddle River, NJ: Prentice Hall, 2000.

[4] STMicroelectronics, *UM1725: Description of STM32F4 HAL and Low-Layer Drivers*, Rev. 8, 2023.

[5] STMicroelectronics, *UM1718: STM32CubeMX for STM32 Configuration and Initialization C Code Generation*, Rev. 5, 2022.

[6] FreeRTOS, *FreeRTOS Kernel Documentation*. [Online]. Available: `https://www.freertos.org/Documentation`. [Accessed: 2026].

[7] ARM Ltd., *Cortex-M4 Technical Reference Manual*, DDI 0439E, 2010.

[8] Texas Instruments, *LM35 Precision Centigrade Temperature Sensors*, Datasheet SNIS159.

[9] STMicroelectronics, *STM32F411RE Datasheet*, DS10314.

Appendix A

Sample Rate-Monotonic Calculation

For task T2 (Sensor) the utilisation contribution is

$$\frac{C_2}{T_2} = \frac{5}{100} = 0.05. \tag{A.1}$$

Summing the four contributions gives $U = 0.126$; the RM bound for $n = 4$ is 0.757; therefore the task set is schedulable.

Appendix B

FreeRTOS Task Source Code

```c
/* T2: Sensor task - priority 3, period 100 ms */
void StartSensorTask(void *argument)
{
    float temperature;
    uint32_t raw;

    for (;;)
    {
        /* Wait for the ADC conversion-complete semaphore (from ISR) */
        if (xSemaphoreTake(xAdcDone, portMAX_DELAY) == pdTRUE)
        {
            raw = HAL_ADC_GetValue(&hadc1);

            /* LM35: 10 mV per degree Celsius, 12-bit ADC, 3.3 V ref */
            temperature = ((raw * 3300.0f) / 4095.0f) / 10.0f;

            /* Publish to the control task; block if the queue is full */
            xQueueSend(xQueueTemp, &temperature, portMAX_DELAY);
        }
    }
}
```

Listing B.1: Temperature sensor task (T2) — producer.

```c
/* T3: Control task - priority 2, period 200 ms */
void StartControlTask(void *argument)
{
    float temperature;
    EventBits_t events;

    for (;;)
```

```
    {
        if (xQueueReceive(xQueueTemp, &temperature, pdMS_TO_TICKS
            (200)))
        {
            if (temperature > TEMP_HIGH_LIMIT)
            {
                HAL_GPIO_WritePin(FAN_GPIO_Port, FAN_Pin,
                    GPIO_PIN_SET);
                HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin,
                    GPIO_PIN_SET);
            }
            else
            {
                HAL_GPIO_WritePin(FAN_GPIO_Port, FAN_Pin,
                    GPIO_PIN_RESET);
                HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin,
                    GPIO_PIN_RESET);
            }

            /* Non-blocking check for a mode-change request from T1
                */
            events = xEventGroupGetBits(xEventGroup);
            if (events & EVT_BUTTON_PRESS)
            {
                automaticMode = !automaticMode;
                xEventGroupClearBits(xEventGroup, EVT_BUTTON_PRESS);
            }
        }
    }
}
```

Listing B.2: Control task (T3) — consumer and actuator driver.

```
/* T4: Logger task - priority 1, period 500 ms */
void StartLoggerTask(void *argument)
{
    char buffer[64];

    for (;;)
    {
        if (xSemaphoreTake(xUartMutex, portMAX_DELAY) == pdTRUE)
        {
            snprintf(buffer, sizeof(buffer),
                     "T=%.2f C  Mode=%s\r\n",
                     lastTemperature,
                     automaticMode ? "AUTO" : "MANUAL");

```

```
            HAL_UART_Transmit(&huart2, (uint8_t *)buffer,
                              strlen(buffer), HAL_MAX_DELAY);

            xSemaphoreGive(xUartMutex);
        }

        osDelay(500);
    }
}
```

Listing B.3: Logging task (T4) — lowest priority, protected by a mutex.

```
/* ISR context: only FromISR APIs may be used here */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    xSemaphoreGiveFromISR(xAdcDone, &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}
```

Listing B.4: ADC interrupt callback — deferred processing via a binary semaphore.

Appendix C

Task Creation Snippet

```
/* Queue, mutex, semaphore and event group */
xQueueTemp    = xQueueCreate(5, sizeof(float));
xUartMutex    = xSemaphoreCreateMutex();
xAdcDone      = xSemaphoreCreateBinary();
xEventGroup   = xEventGroupCreate();

/* Task creation: name, stack, parameter, priority, handle */
xTaskCreate(StartButtonTask,  "BTN",  128, NULL, 4, NULL);
xTaskCreate(StartSensorTask,  "TEMP", 256, NULL, 3, NULL);
xTaskCreate(StartControlTask, "CTRL", 256, NULL, 2, NULL);
xTaskCreate(StartLoggerTask,  "LOG",  512, NULL, 1, NULL);

/* Start the scheduler - must never return */
vTaskStartScheduler();
```

Listing C.1: Task and primitive creation inside `MX_FREERTOS_Init()`.

Appendix D

Datasheets and Additional Resources

- LM35 Precision Centigrade Temperature Sensor — Texas Instruments SNIS159.
- STM32F411RE Datasheet — STMicroelectronics DS10314.
- FreeRTOS API Reference — `https://www.freertos.org`.
- CMSIS-RTOS v2 API Reference — ARM Ltd.